



北京邮电大学

软件安全

北京邮电大学网络空间安全学院

张淼

zhangmiao@bupt.edu.cn

北京邮电大学

BEIJING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS



第七讲 指针安全

- 函数指针攻击

- 对象指针攻击

- 虚函数攻击

- SEH攻击

- Windows内存安全机制概述

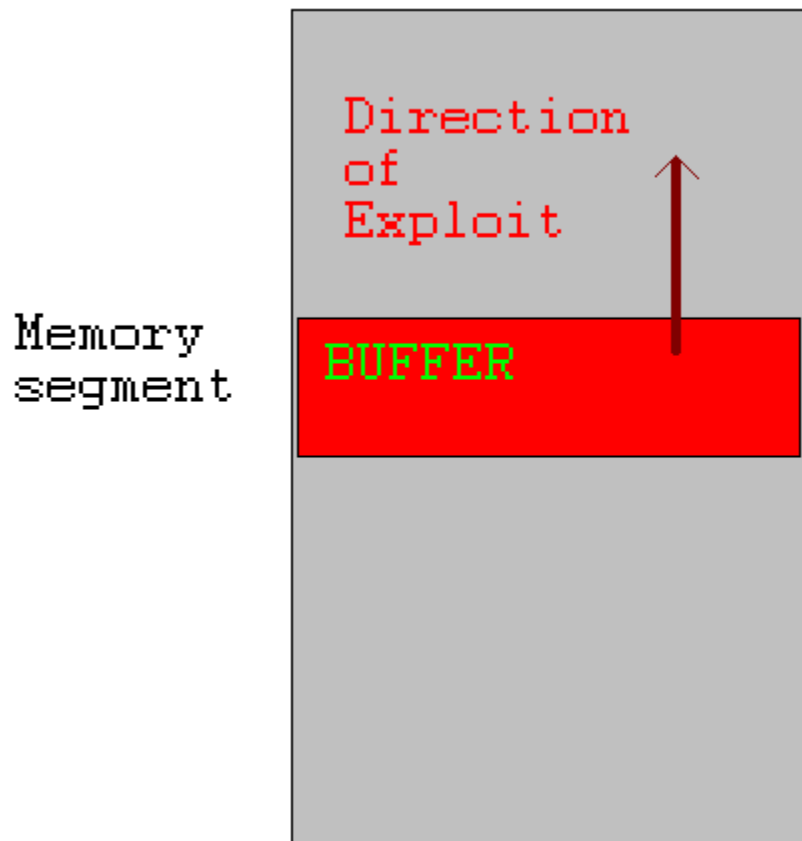


指针安全

- 指针安全是通过修改指针值来利用程序漏洞的方法的统称
 - ➔ 可以通过覆盖函数指针将程序的控制权转移到攻击者提供的shellcode
 - ➔ 对象指针也可以被修改，从而执行任意代码



指针安全



- 缓冲区溢出覆写指针条件：
 - 缓冲区与目标指针必须分配在同一个段内
 - 缓冲区必须位于比目标指针更低的内存地址处
 - 该缓冲区必须是界限不充分的，因此容易被缓冲区溢出利用



指针安全

- **UNIX**可执行文件包含**data**段和**BSS**段
- **data**段包含了所有已初始化的全局变量和常数
- **BSS** (**B**lock **S**tarted by **S**ymbols) 段包含了所有未初始化的全局变量
- 已初始化的全局变量和未初始化变量分开是为了让汇编器不将未初始化的变量内容写入目标文件



指针安全

```
1. static int GLOBAL_INIT = 1;          /* data segment, global */
2. static int global_uninit;             /* BSS segment, global */
3.
4. void main(int argc, char **argv) {    /* stack, local */
5.     int local_init = 1;                /* stack, local */
6.     int local_uninit;                  /* stack, local */
7.     static int local_static_init = 1; /* data seg, local */
8.     static int local_static_uninit;    /* BSS segment, local */
        /* storage for buff_ptr is stack, local */
        /* allocated memory is heap, local */
9. }
```



指针安全

```
void good_function(const char *str) {  
    //do something  
}  
  
int main(int argc, char **argv) {  
    if (argc !=2){  
        printf("Usage: prog_name <string1>\n");  
        exit(-1);  
    }  
    static char buff [BUFSIZE];  
    static void (*funcPtr)(const char *str);  
    funcPtr = &good_function;  
    strncpy(buff, argv[1], strlen(argv[1]));  
    (void) (*funcPtr) (argv[2]);  
    return 0;  
}
```



指针安全

- 程序存在漏洞，可被缓冲区溢出利用
- 缓冲区和函数指针都未初始化，因此存在于**BSS**段



指针安全

```
void good_function(const char *str) {
    //do something
}

int main(int argc, char **argv) {
    if (argc !=2){
        printf("Usage: prog_name <string1>\n");
        exit(-1);
    }
    static char buff [BUFSIZE];
    static void (*funcPtr)(const char *str);
    funcPtr = &good_function;
    strncpy(buff, argv[1], strlen(argv[1]));
    (void) (*funcPtr) (argv[2]);
    return 0;
}
```



函数指针举例

```
1. void good_function(const char *str) {...}
2. void main(int argc, char **argv) {
3.   static char buff[BUFSIZE];
4.   static void (*funcPtr)(const char *str);
5.   funcPtr = &good_function;
6.   strncpy(buff, argv[1], strlen(argv[1]));
7.   (void)(*funcPtr)(argv[2]);
8. }
```

静态字符数
组buff

funcPtr都是未初始化的，
并且存储于BSS段



函数指针举例

```
1. void good_function(const char *str) {...}
2. void main(int argc, char **argv) {
3.     static char buff[BUFSIZE];
4.     static void (*funcPtr)(const char *str);
5.     funcPtr = &good_function;
6.     strncpy(buff, argv[1], strlen(argv[1]));
7.     (void)(*funcPtr)(argv[2]);
8. }
```

当argv[1]的
长度大于
BUFSIZE的
时候，就会
发生缓冲区
溢出



函数指针举例

```
1. void good_function(const char *str) {...}
2. void main(int argc, char **argv) {
3.   static char buff[BUFSIZE];
4.   static void (*funcPtr)(const char *str);
5.   funcPtr = &good_function;
6.   strncpy(buff, argv[1], strlen(argv[1]));
7.   (void)(*funcPtr)(argv[2]);
8. }
```

当程序执行到funcPtr标识的函数的时候，shellcode将会取代good_function()得以执行



对象指针举例

```
void foo(void * arg, size_t len)    {  
    char buff[100];  
    long val = ...;  
    long *ptr = ...;  
    memcpy(buff, arg, len);  
    *ptr = val;  
    ...  
    return;  
}
```

缓冲区容易被漏洞利用

在溢出缓冲区后，攻击者可以覆写ptr和val

会发生任意内存写



对象指针

- “任意内存写”可以改变程序控制流
- 如果指针长度等于重要的数据结构长度，任意内存写会更容易

→ Intel 32 Architectures:

➤ $\text{sizeof}(\text{void}^*) = \text{sizeof}(\text{int}) = \text{sizeof}(\text{long}) = 4\text{B}$



修改指令指针

- Instruction Counter (IC) (a.k.a Program Counter (PC))存储了将要执行的下一条指令地址
 - Intel: EIP 寄存器
- IC不能被直接访问
- IC在顺序执行代码时递增，也可以由控制转移指令间接修改
 - jmp
 - Conditional jumps
 - call
 - ret



修改指令指针

```
int _tmain(int argc, _TCHAR* argv[]) {  
    if (argc !=1){  
        printf("Usage: prog_name\n");  
        exit(-1);  
    }  
    static void (*funcPtr)(const char *str);  
    funcPtr = &good_function;  
    (void) (*funcPtr) ("hi ");  
    good_function("there!\n");  
    return 0;  
}
```

使用指针调用good function

直接调用good function



修改指令指针

```
void) (*funcPtr) ("hi ");
```

```
00424178 mov esi, esp
```

```
0042417A push offset string "hi" (46802Ch)
```

```
0042417F call dword ptr [funcPtr (478400h)]
```

```
00424185 add esp, 4
```

```
00424188 cmp esi, esp
```

第一个调用good
function

机器码

ff 15 00 84 47 00

最后4个字节包含了被
调用函数的地址

```
good_function("there!\n");
```

```
0042418F push offset string "there!\n" (468020h)
```

```
00424194 call good_function (422479h)
```

```
00424199 add esp, 4
```

第二个调用good
function

机器码

e8 e0 e2 ff ff

最后4字节被调用函数
的相对偏移量



修改指令指针

- 静态调用对于函数地址使用立即数
 - 指令中地址被编码
 - 计算地址，然后放入IC
 - 不改变执行指令，IC不会改变
- 通过函数指针的调用是间接引用
 - IC的下一个值，存储在内存中，其可以被改变



修改指令指针

- 控制IC使得攻击者可以选择要执行的代码
 - ➔ 攻击者能够任意写的话，很容易
 - ➔ 间接的函数引用与无法在编译期间决定的函数调用可以被利用，从而使程序的控制权转移到任意代码



第七讲 指针安全

- 函数指针攻击

- 对象指针攻击

- 虚函数攻击

- SEH攻击



虚函数攻击

多态是面向对象的一个重要特性，在C++中，这个特性主要靠对虚函数的动态调用来实现。

由于我们的重点是讲解利用虚函数进行攻击，所以对虚函数和动态联编等概念不太了解的同学可以课下学习一下。

在仅仅关注漏洞利用的前提下，我们可以简单地把虚函数和虚表理解为一下几个要点。

1) C++类的成员函数在声明时，若使用关键字 **virtual** 进行修饰，则被称为虚函数。

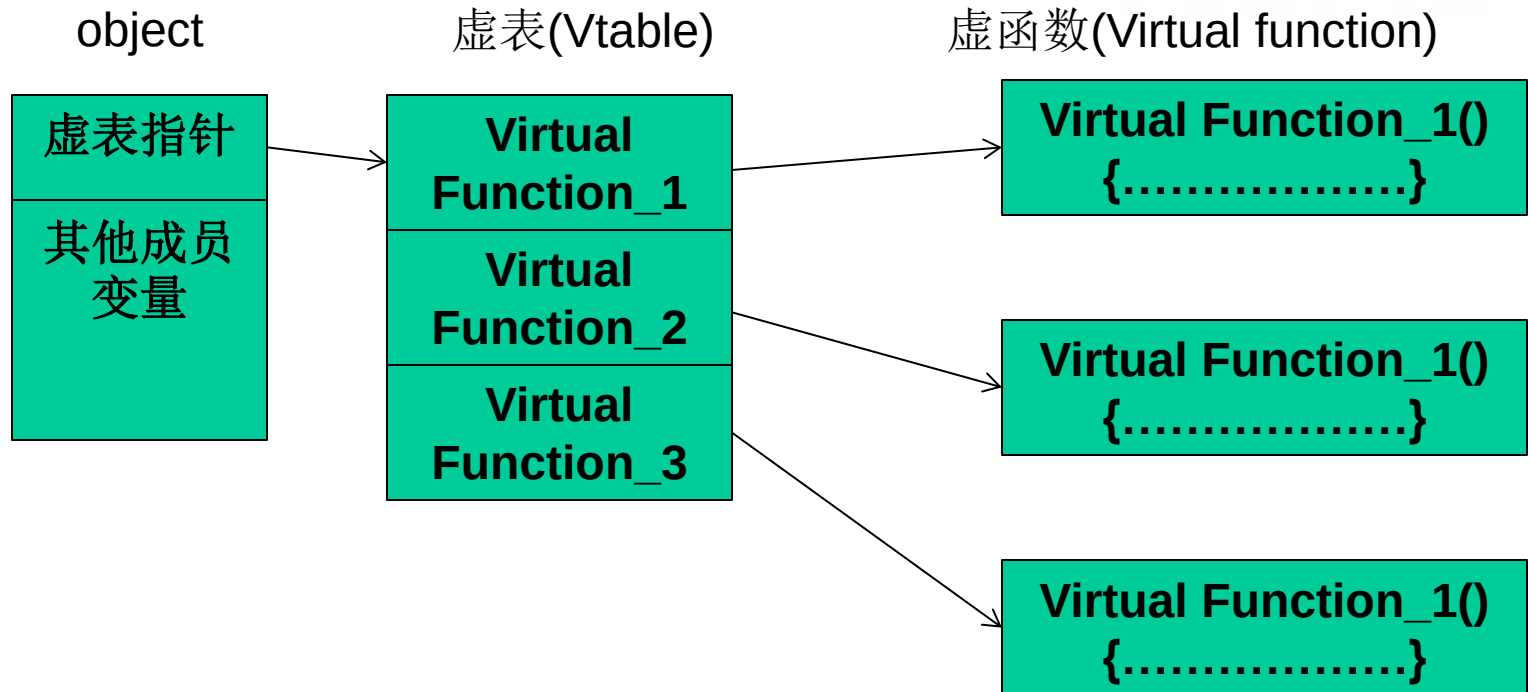


虚函数攻击

- 2) 一个类中可能有很多个虚函数。
- 3) 虚函数的入口地址被统一保存在虚表(Vtable)中。
- 4) 对象在使用虚函数时，先通过虚表指针找到虚表，然后从虚表中取出最终的函数入口地址进行调用。
- 5) 虚表指针保存在对象的内存空间中，紧接着虚表指针的是其他成员变量。
- 6) 虚函数只有通过对象指针的引用才能显示出其动态调用的特性。



虚函数攻击



虚函数的实现



虚函数攻击

我们来写一个利用虚函数执行shellcode的程序

```
char[] shellcode=".....";  
class vf  
{  
    public:  
    char buf[200];  
    virtual void test(void)  
    {  
        cout<<"Class Vtable::test()"<<endl;  
    }  
};  
vf overflow, *p;
```

这段程序声明了一个类，具有buf和虚函数test，声明它的实例overflow和指针p。



虚函数攻击

```
void main(void)
{
    LoadLibrary("user32.dll");
    char * p_vtable;
    p_vtable=overflow.buf-4;//point to virtual
table
    __asm int 3
    //reset fake virtual table to 0x0042E430
    //the address may need to adjusted via
runtime debug
```



虚函数攻击

```
p_vtable[0]=0x30;  
p_vtable[1]=0xE4;  
p_vtable[2]=0x42;  
p_vtable[3]=0x00;  
strcpy(overflow.buf,shellcode1);//set fake  
        virtual function pointer  
p=&overflow;  
p->test();  
}
```



虚函数攻击

- 1) 虚表指针位于成员变量char buf[200]之前，程序中通过p_vtable=overflow.buf-4定位到这个指针。
- 2) 修改虚表指针指向缓冲区的0x0042E430处（第一次得不到，需要通过调试得到）。
- 3) 程序执行到p->test()时，将按照伪造的虚函数指针去0x0042E430寻找虚表，这里正好是缓冲区里shellcode的末尾。在这里填上shellcode的起始位置0x0042E35C作为伪造的虚函数入口地址，程序将最终跳去执行shellcode。



虚函数攻击



利用虚表的示意图



虚函数攻击

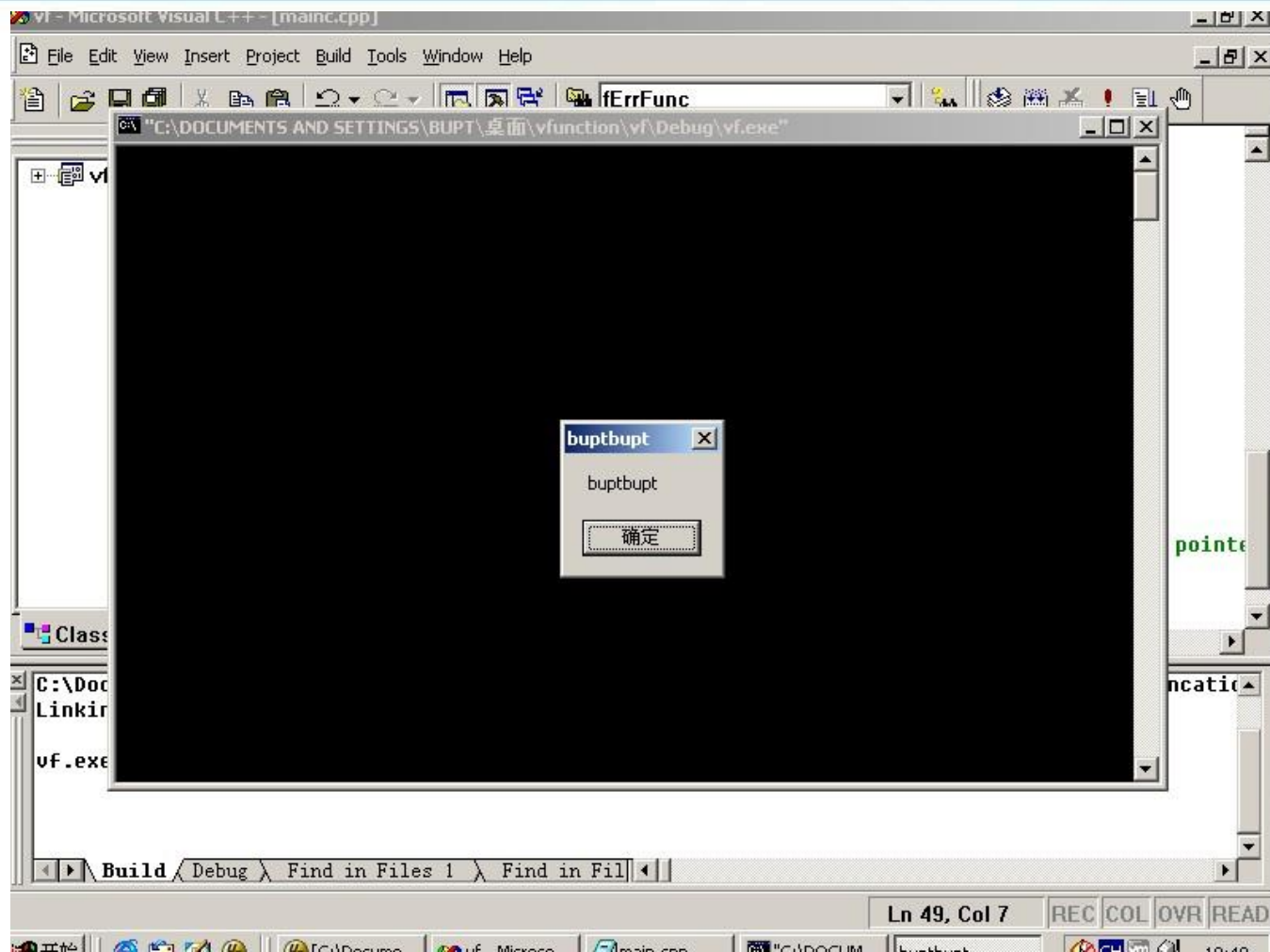
0012FF20	00000000	
0012FF24	00000000	
0012FF28	0042E35C	dest = vf.0042E35C
0012FF2C	0042AE50	src = "哈哈哈哈哈哈"
0012FF30	00132588	
0012FF34	00132580	

Int3触发后我们可以在strcpy函数上看到shellcode的起始地址0x0042E35C,然后可以计算出shellcode的末尾后四个字节地址是0X0042E430。

得到这两个地址后，修改程序，将vtable和shellcode那两个部分进行修改，然后就可以运行程序了



虚函数攻击



运行程序后可以看到这个结果。



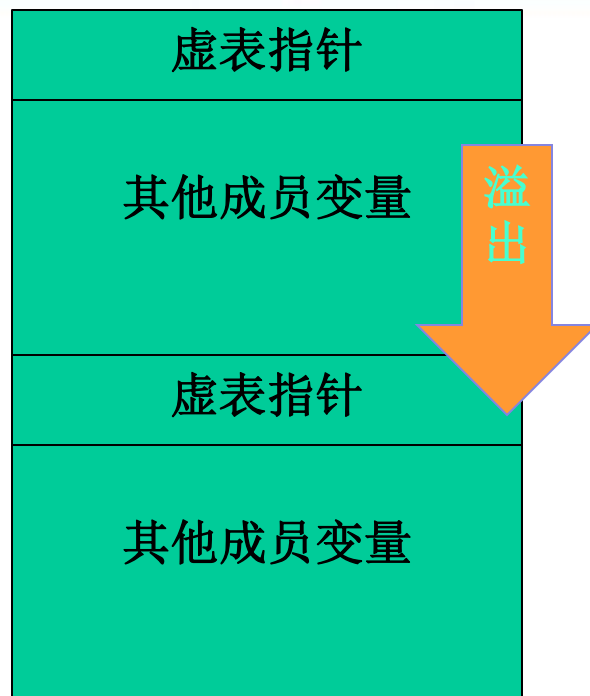
虚函数攻击

由于虚表指针位于成员变量之前，溢出只能向后覆盖数据，所以很可惜这种利用方式在“栈溢出”场景下有一定局限性。

当然，如果内存中存在多个对象且能够溢出到下一个对象空间中去，“连续性覆盖”还是有攻击的机会的，比如下图这种情况。



虚函数攻击



溢出邻接对象的虚表



虚函数攻击

说到这里，大家应该明白所谓的虚函数、面向对象在指令层次上和C语言是没有质的区别的，以漏洞利用的眼光来看这些东西，其实都是指针。



第七讲 指针安全

- 函数指针攻击

- 对象指针攻击

- 虚函数攻击

- SEH攻击



S.E.H概述

操作系统或程序在运行时，难免会遇到各种各样的错误，如除0、非法内存访问、文件打开错误、内存不足、磁盘读写错误、外设操作失败等。



S.E.H概述

为了保证系统在遇到错误时不至于崩溃，仍能够见状稳定地继续运行下去，Windows会对运行在其中的程序提供一次补救的机会来处理错误，这种机制就是异常处理机制。



S.E.H概述

S.E.H即异常处理结构体(Structure Exception Handler)

它是Windows异常处理机制所采用的重要数据结构。

每个S.E.H包含两个DWORD指针：S.E.H链表指针和异常处理函数句柄，共8个字节，如图所示。



S.E.H概述

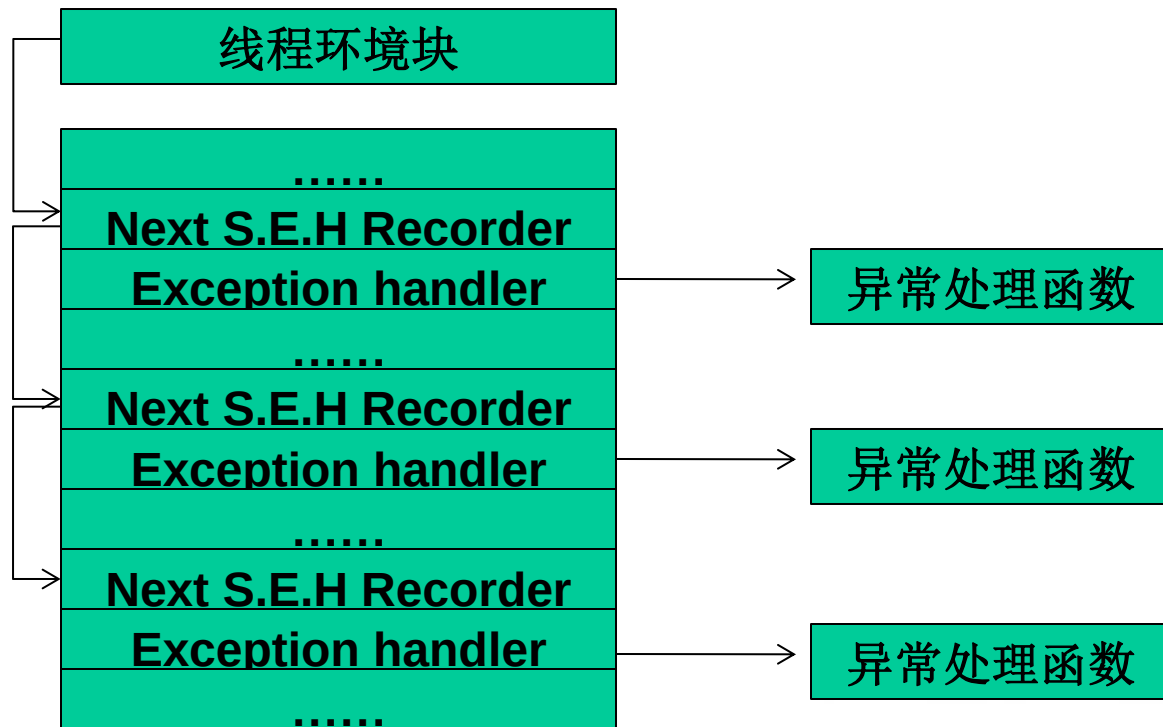
DWORD:Next S.E.H recorder
DWORD:Exception handler

S.E.H结构体



S.E.H概述

作为对S.E.H的初步了解，我们现在需要知道以下几个要点，S.E.H链表如下图所示。





S.E.H概述

- 1) S.E.H结构体存放在系统栈中。
- 2) 当线程初始化时，会自动向栈中安装一个S.E.H，作为线程默认的异常处理。
- 3) 如果程序源代码中使用了__try{}__except{}或者Assert宏等异常处理机制，编译器将最终通过向当前函数栈帧中安装一个S.E.H来实现异常处理。
- 4) 栈中一般会同时存在多个S.E.H。



S.E.H概述

- 5) 栈中的多个S.E.H通过链表指针在栈内由栈顶向栈底串成单向链表，位于链表最顶端的S.E.H通过T.E.B(线程环境块)0字节偏移处的指针标识。
- 6) 当异常发生时，操作系统会中断程序，并首先从T.E.B的0字节偏移处取出距离栈顶最近的S.E.H，使用异常处理函数句柄所指向的代码来处理异常。



S.E.H概述

- 7) 当离“事故现场”最近的异常处理函数运行失败时，
将顺着S.E.H链表依次尝试其他的异常处理函数。

- 8) 如果程序安装的所有异常处理函数都不能处理，系统
将采用默认的异常处理函数。通常，这个函数会弹
出一个对话框，然后强制关闭程序。



S.E.H概述

上面所叙述的只是对S.E.H进行了一下简单的概述，省略了很多细节。

例如，系统对异常处理函数的调用可能不止一次；
对同一个函数内的多个__try或者嵌套的__try需要进行S.E.H展开操作；
执行异常处理函数前会进行若干判定操作等等。



S.E.H概述

从程序设计的角度来讲，S.E.H就是在系统关闭程序之前，给程序一个执行预先设定的回调函数的机会。

大概明白了S.E.H的工作原理之后，同学们能否设计出利用S.E.H攻击的思路呢？



S.E.H概述

我们一步一步来思考。

首先，S.E.H存放在栈内，我们学过的栈溢出漏洞可能会派上用场。

其次，我们可以精心制造出溢出数据来把S.E.H的异常处理的函数地址替换为shellcode的起始地址。



S.E.H概述

再次，溢出后错误的栈帧或堆块数据往往会触发异常，或者我们能查到哪些能触发异常的片段。

最后，当Windows开始处理溢出后的异常时，会调用什么呢？对，就是我们的 shellcode，它会把 shellcode 当作异常函数来处理异常。



S.E.H概述

以上就是我们利用S.E.H来进行攻击的思路。

接下来我们来实验一下，来在自己写的有漏洞的小程序上，在栈溢出中利用S.E.H执行shellcode。



S.E.H概述

我们先来看一下我们的源代码

```
#include <windows.h>
```

```
char shellcode[]="\x90\x90...";// 由于我们要得到
```

```
//shellcode起始位置, S.E.H地址等信息, 所以我们
```

```
//先用0x90来进行填充
```




S.E.H概述

```
DWORD MyExceptionHandler(void)
```

```
{
```

```
    printf("got an exception,press Enter to kill  
process!\n");
```

```
    getchar();
```

```
    ExitProcess(1);
```

```
}//这是我们自己编写的异常处理函数
```



S.E.H概述

```
void test(char* input)
```

```
{
```

```
    char buf[200];
```

```
    int zero=0;
```

```
    __asm int 3 //int 3指令可以中断进程开始调试
```

```
    __try
```

```
{
```

```
    strcpy(buf,input);//产生栈溢出。未完，接下页
```



S.E.H概述

```
    zero=4/zero;//除0产生异常  
}  
__except(MyExceptionHandler){  
}  
int main(){  
    test(shellcode);  
    return 0;  
}
```



S.E.H概述

对代码进行一下简要的解释。

- 1) 函数test中存在典型的栈溢出漏洞。
- 2) `__try{}`会在test的函数栈帧中安装一个S.E.H结构。
- 3) `__try`中的除0操作会产生一个异常
- 4) 当strcpy操作没有产生溢出时，除0操作的异常将最终被MyExceptionHandler函数处理。



S.E.H概述

对代码进行一下简要的解释。

- 5) 当strcpy操作产生溢出，并精确地将栈帧中的S.E.H异常处理句柄修改为shellcode的入口地址时，操作系统将会错误地使用shellcode去处理除0异常，代码植入成功。



S.E.H概述

对代码进行一下简要的解释。

- 6) 此外，异常处理机制会检测进程是否处于调试状态。如果直接用调试器加载程序，异常处理会进入调试状态下的处理流程。因此我们在代码中加入断点 `__asm int 3`，让进程中断，然后用调试器attach的方法进行调试



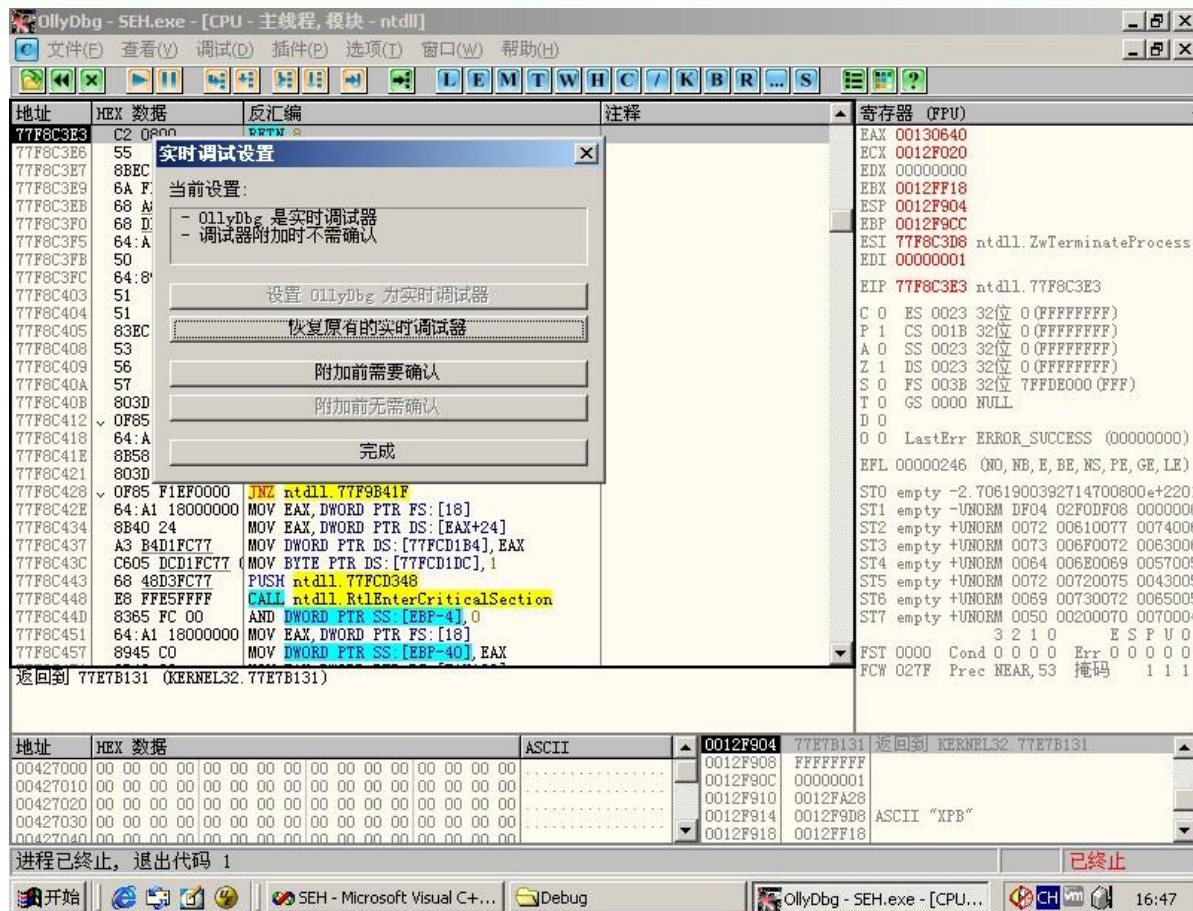
S.E.H概述

	推荐使用的环境	备注
操作系统	Win 2000	虚拟机实体机均可
编译器	Vc 6.0	
编译选项	默认编译选项	
Build版本	Release版本	必须使用release版本进行调试

关于实验环境必须要说明，只在win2000下才能进行实验，winXP SP2和Win 2003以及之后的版本都对SEH进行了优化，会导致实验的失败，具体优化细节我们可能会在以后的课上进行讲述



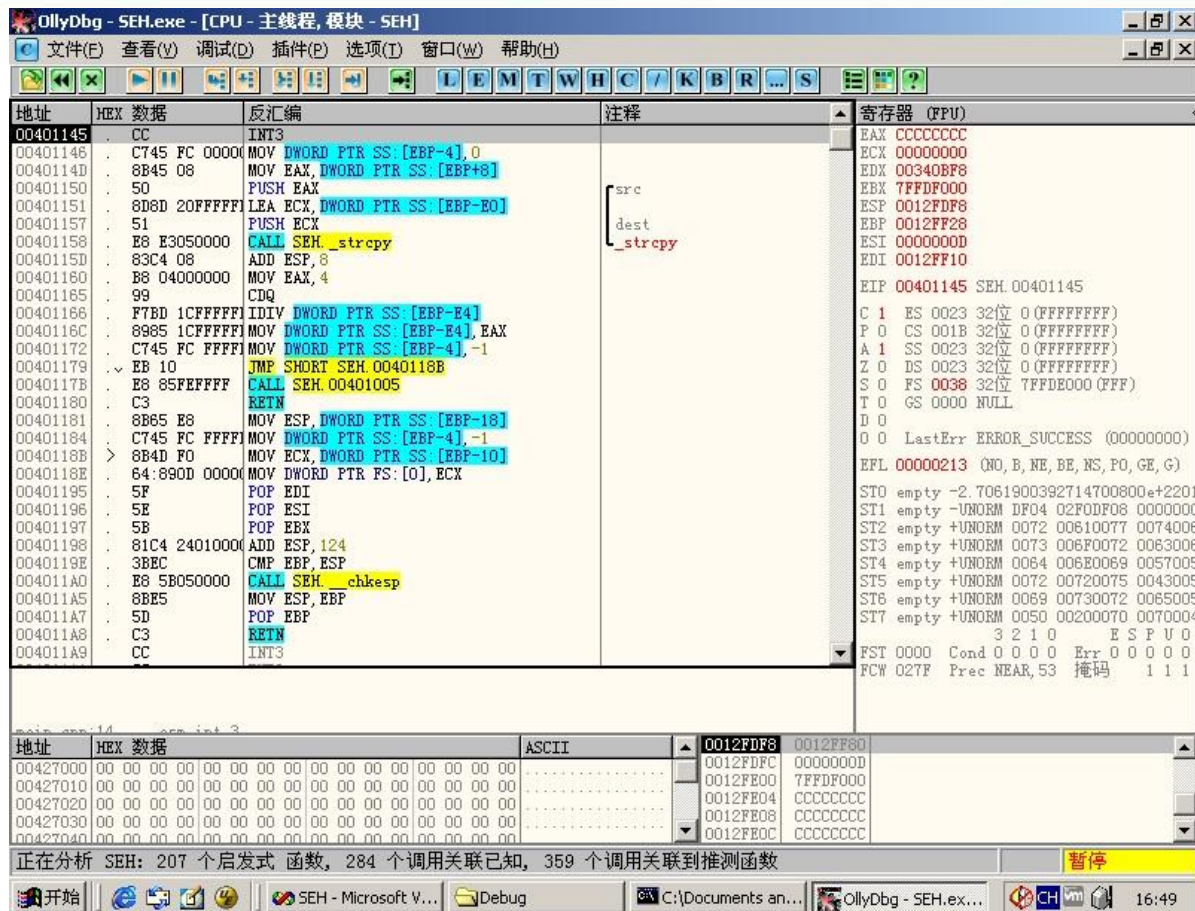
S.E.H概述



为了能触发int 3断点时启动OllyDbg，我们选择选项中的实时调试设置
选择设置OllyDbg为实时调试器，然后当我们运行exe文件时，int 3 断点
触发后就会启动OllyDbg



S.E.H概述



成功在 int3 上启动了 Ollydbg，有可能启动时要求设置 UDD 和 Plugin 目录，我们设置一下即可



S.E.H概述

0012FDF0	0012FE48	dest = 0012FE48
0012FDF4	00427B40	src = "哈哈哈哈"
0012FDF8	0012FF80	
0012FDFC	00000000	
0012FE00	7FFDF000	
0012FE04	CCCCCCCC	

暂停

在执行strcpy函数之前，我们可以看到shellcode的起始地址

0012FE48



S.E.H概述

0012FE48	90909090	
0012FE4C	90909090	
0012FE50	CCCCC00	
0012FE54	CCCCCCCC	
0012FE58	CCCCCCCC	
0012FE5C	CCCCCCCC	

执行完strcpy后，确认0012FE48是我们shellcode的起始



S.E.H概述



地址	SE处理程序
0012FF18	SEH.__except_handler3
0012FFB0	SEH.__except_handler3
0012FFE0	KERNEL32.77E813FD

我们点击查看菜单中的S.E.H链，我们会看到这个S.E.H链的情况，我们主要针对第一个地址。



S.E.H概述

0012FF10	0012FDF8	
0012FF14	0012FA40	
0012FF18	0012FFB0	指向下一个 SEH 记录的指针
0012FF1C	00401928	SE处理程序
0012FF20	00425058	SEH. 00425058
0012FF24	FFFFFFFF	

我们去看一下那个地址的记录，果然，是一个指向下一个SEH的指针，接着是异常处理程序，我们只需要把0012FF1C这个地址的内容改成我们的shellcode起始地址就可以了



S.E.H概述

我们查得了shellcode的起始地址为

0x0012FE48

第一个S.E.H地址为

0x0012FF18(指向下一个S.E.H的指针)

0x0012FF1C(异常处理地址)

那么我们的shellcode应该有

$0x0012FF1C - 0x0012FE48 = 212$ 的字节进行填

充



S.E.H概述

我们还使用我们曾经使用过的buptbupt弹出对话框(静态地址版, 不是jmp esp的那个版本)的那个shellcode内容, 作为我们shellcode的核心内容。

剩下的空间我们用0x90来补齐至212字节, 然后我们在213-216字节使用0x0012FE48填充(不要忘了内存的大端小端的问题)。



S.E.H概述

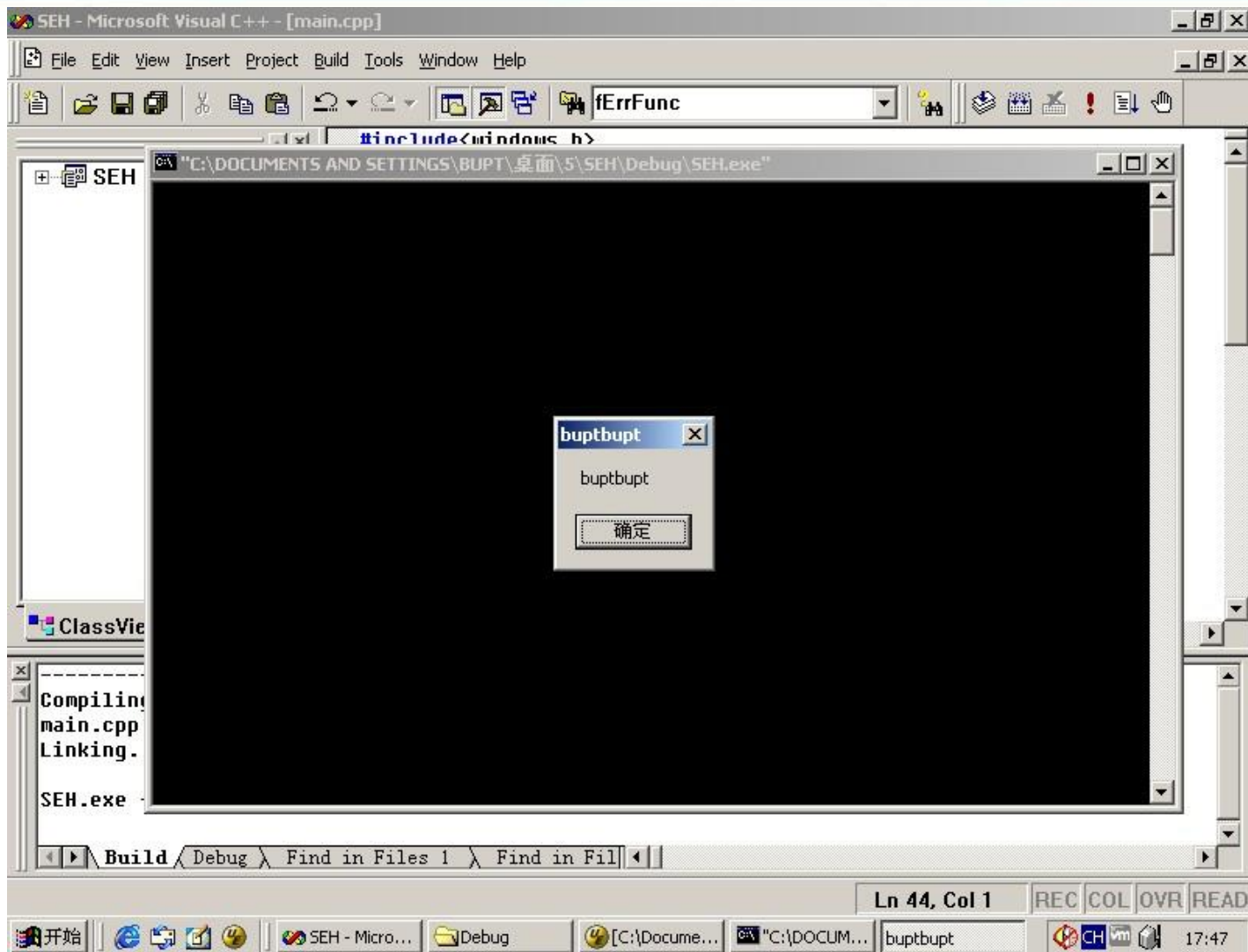
我们再来修改一下源代码，在main函数中加入
LoadLibrary("user32.dll");//为了我们能够顺利调用出
//对话框

取消__asm int 3

然后我们直接启动程序



S.E.H概述





S.E.H概述

我们看到我们的shellcode已经被执行，但是我们点击确定却没有反应，因为我们的shellcode已经被当作系统异常处理来进行了，所以会有所不同。



S.E.H概述

在S.E.H方面，微软也做了一些改进，比如SafeSEH和SEHOP。

在Windows XP SP2及后续版本的操作系统中，微软引入了著名的S.E.H校验机制SafeSEH。

SafeSEH的原理很简单，在程序调用异常处理函数之前，对要调用的异常处理函数进行一系列的有效性校验，当发现异常处理函数不可靠时将终止异常处理函数的调用。



S.E.H概述

SafeSEH需要操作系统和编译器的双重支持，二者缺一都会降低SafeSEH的保护能力。

SafeSEH的保护措施：

- 1) 检查异常处理链是否位于当前程序的栈中。
如果不在当前栈中，程序将终止异常处理函数的调用。
- 2) 检查异常处理函数指针是否指向当前程序的栈中，如果指向当前栈中，程序将终止异常处理函数的调用。



S.E.H概述

3) 在前面两项检查都通过后，程序调用一个全新的函数RtlIsValidHandler()，来对异常处理函数的有效性进行验证，稍后我们会详细介绍RtlIsValidHandler()函数。



S.E.H概述

首先该函数判断异常处理函数地址是不是在加载模块的内存空间，如果属于加载模块的内存空间，校验函数将依次进行如下校验。

1) 判 断 程 序 是 否 设 置 了
IMAGE_DLLCHARACTERISTICS_NO_SEH标识。
如果设置了这个标识，这个程序内的异常会被忽略。
所以当这个标识被设置时，直接返回失败。



S.E.H概述

2) 检验程序是否包含安全S.E.H表。如果程序包含安全S.E.H表，则将当前的异常处理函数地址与该表进行匹配，匹配成功则返回校验成功，匹配失败则返回校验失败。

3) 判断程序是否设置Ilonly标识。如果设置了这个标识，说明该程序只包含.NET编译人中间语言，函数直接返回校验失败。



S.E.H概述

4) 判断异常处理函数地址是否位于不可执行页上。当异常处理函数地址位于不可执行页上时，校验函数将检测DEP是否开启，如果系统未开启DEP则返回校验成功，否则程序抛出访问违例的异常。



S.E.H概述

如果异常处理函数的地址没有包含在加载模块的内存空间，校验函数将直接进行DEP相关检测，函数依次进行如下校验。

1) 判断异常处理函数地址是否位于不可执行页上。但异常处理函数位于不可执行页上时，校验函数将检测DEP是否开启，如果系统未开启DEP则返回校验成功，否则抛出访问违例异常。



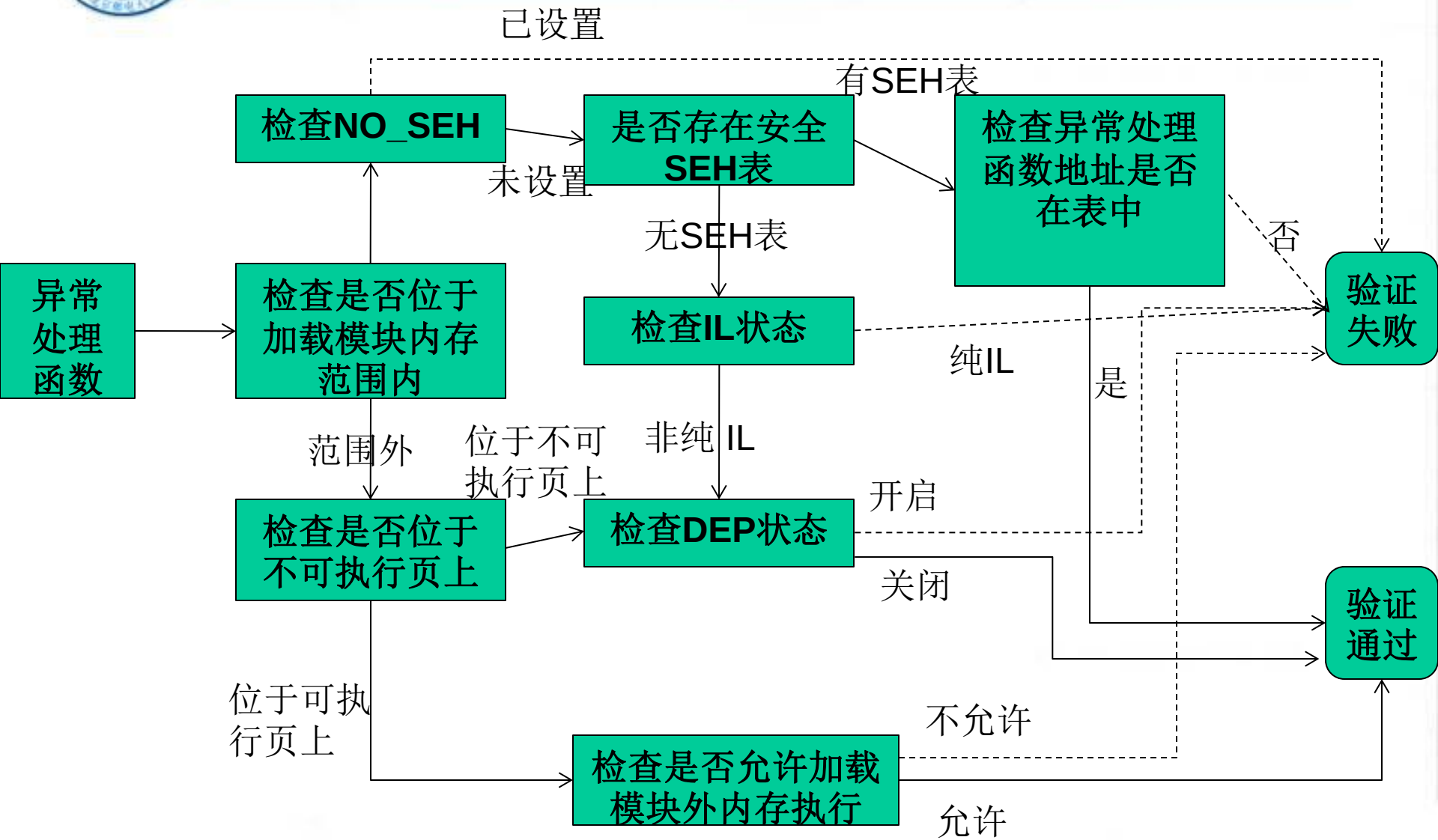
S.E.H概述

2) 判断系统是否允许跳转到加载模块的内存空间外执行，如果允许则返回校验成功，否则返回校验失败。

那我们来看看下面的流程图：



S.E.H概述



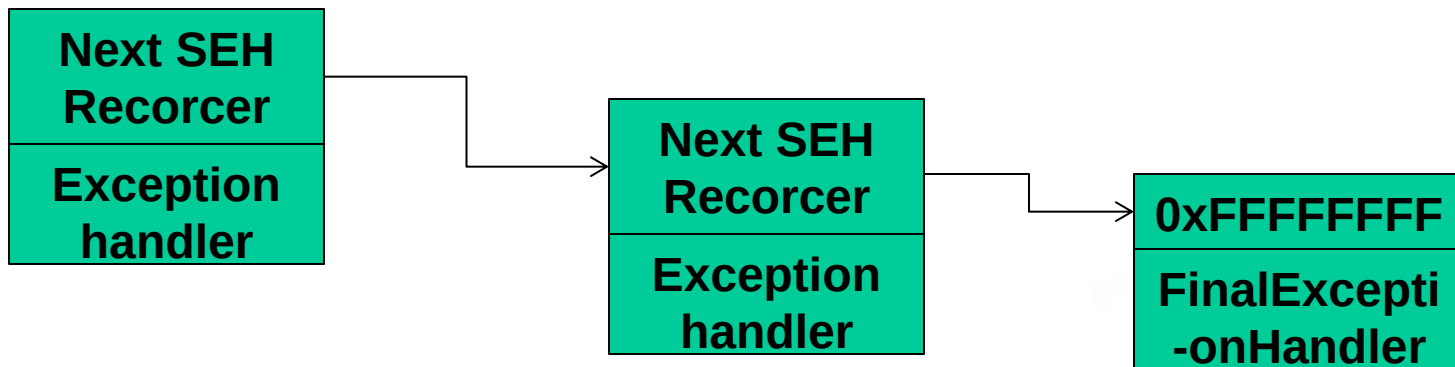


S.E.H概述

SEHOP是一种比SafeSEH更为严厉的保护机制。

目前Vista SP1和Windows 7支持SEHOP。

我们来看看SEHOP是干什么的。我们知道S.E.H函数是以单链表的形式存放在栈中的，末端是默认异常处理。



典型的S.E.H链



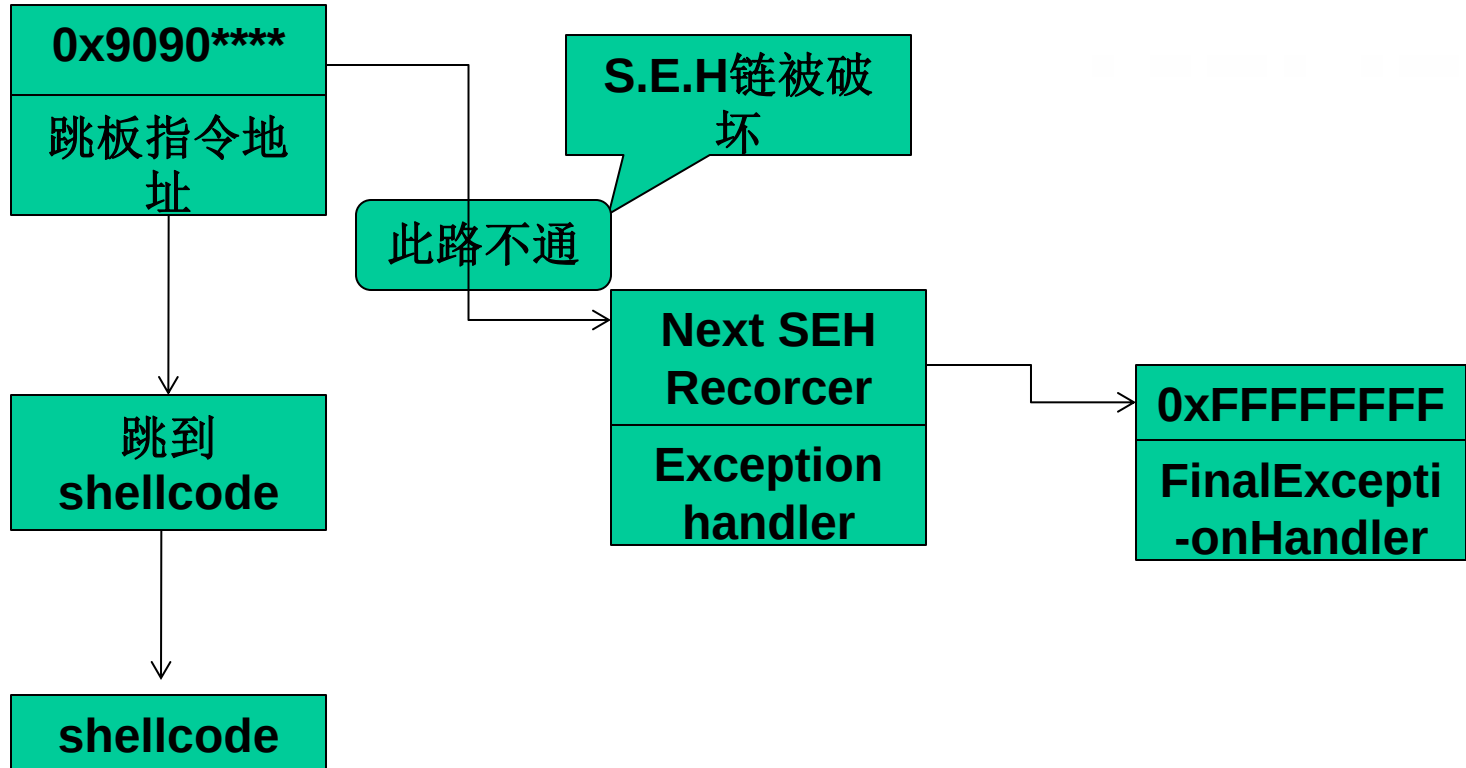
S.E.H概述

SEHOP的核心任务就是检查这条链的完整性，在程序转入异常处理前SEHOP会检查S.E.H链上最后一个异常处理函数是否为系统固定的终极异常处理函数。

如果是，说明这个链没有被破坏，可以去执行当前的异常处理函数；如果检测不是，则说明被破坏，程序不会去执行当前的异常处理函数。



S.E.H概述



典型的S.E.H溢出过程



Windows安全机制概述

● Windows内存安全机制概述



Windows安全机制概述

在过去的十年中，微软在提高操作系统的安全性方面做着不懈的努力。

从Windows 98到Windows XP,从Windows XP到Windows Vista,再到Windows 7、8、10,每个新版本的发布都会带来安全性的质的飞跃。



Windows安全机制概述

从普通用户角度来看，微软在安全方面逐步做了如下几点增强。

1) 增加了Windows安全中心，提醒用户使用杀毒软件、防火墙，以及下载最新的安装补丁等。

2) 为Windows添加PC端的防火墙。

3) 未经用户允许，大多数的Web弹出窗口和ActiveX控件安装将被禁止。



Windows安全机制概述

4) IE中增加了筛选仿冒网站功能, 具有了钓鱼网站过滤器的功能。

5) 添加UAC(User Account Control, 用户账户控制)机制, 可以防止恶意软件和间谍软件在未经许可的情况下在计算机上安装或对计算机进行更改。

6) 集成了Windows Defender, 可以帮助阻止、控制和删除间谍软件以及其他潜在的恶意软件。



Windows安全机制概述

在这些安全功能的保护下，我们操作系统的安全性大大提高了，但是微软所做的工作还远远不止于此。在内存保护方面也做了相当多的工作，下面我们来看看微软是如何提高内存保护的安全性的。



Windows安全机制概述

1) 使用**GS编译技术**，在函数返回地址前首先检测**Security Cookie**是否被覆盖，从而把针对操作系统的栈溢出变得非常困难。

针对缓冲区溢出时覆盖返回函数地址这一特征，微软在编译程序时使用了一个安全编译选项—**GS**，在**vs2003**以及之后的版本中默认启用了这个编译选项。



Windows安全机制概述

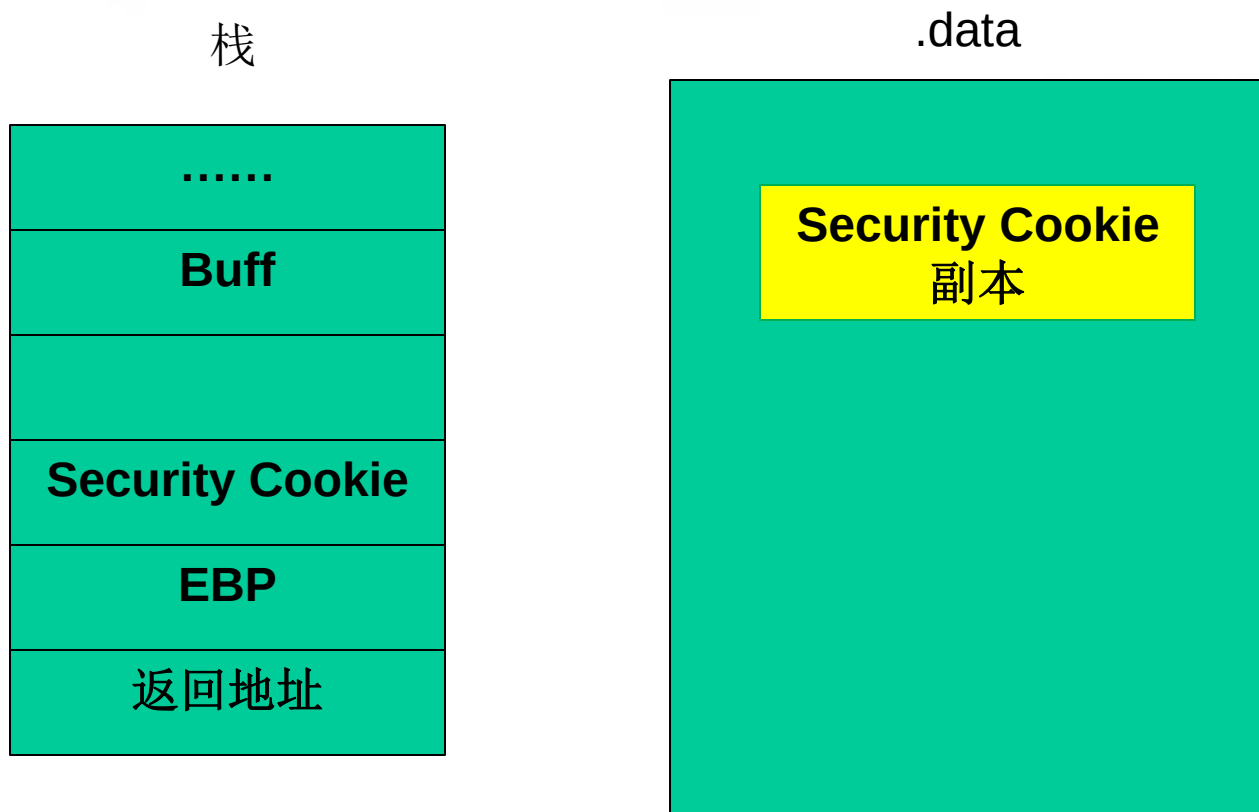
GS编译选项为每个函数调用增加了一些额外的数据和操作，用以检测栈中的溢出。

在所有函数调用发生时，向栈帧内压入一个额外的DWORD，这个随机数被称做“canary”，但如果使用IDA反汇编的话，我们会看到IDA会将这个随机数标注为“Security Cookie”。

Security Cookie位于EBP之前，系统还将在.data的内存区域存放一个Security Cookie的副本。



Windows安全机制概述



GS保护机制下的内存布局



Windows安全机制概述

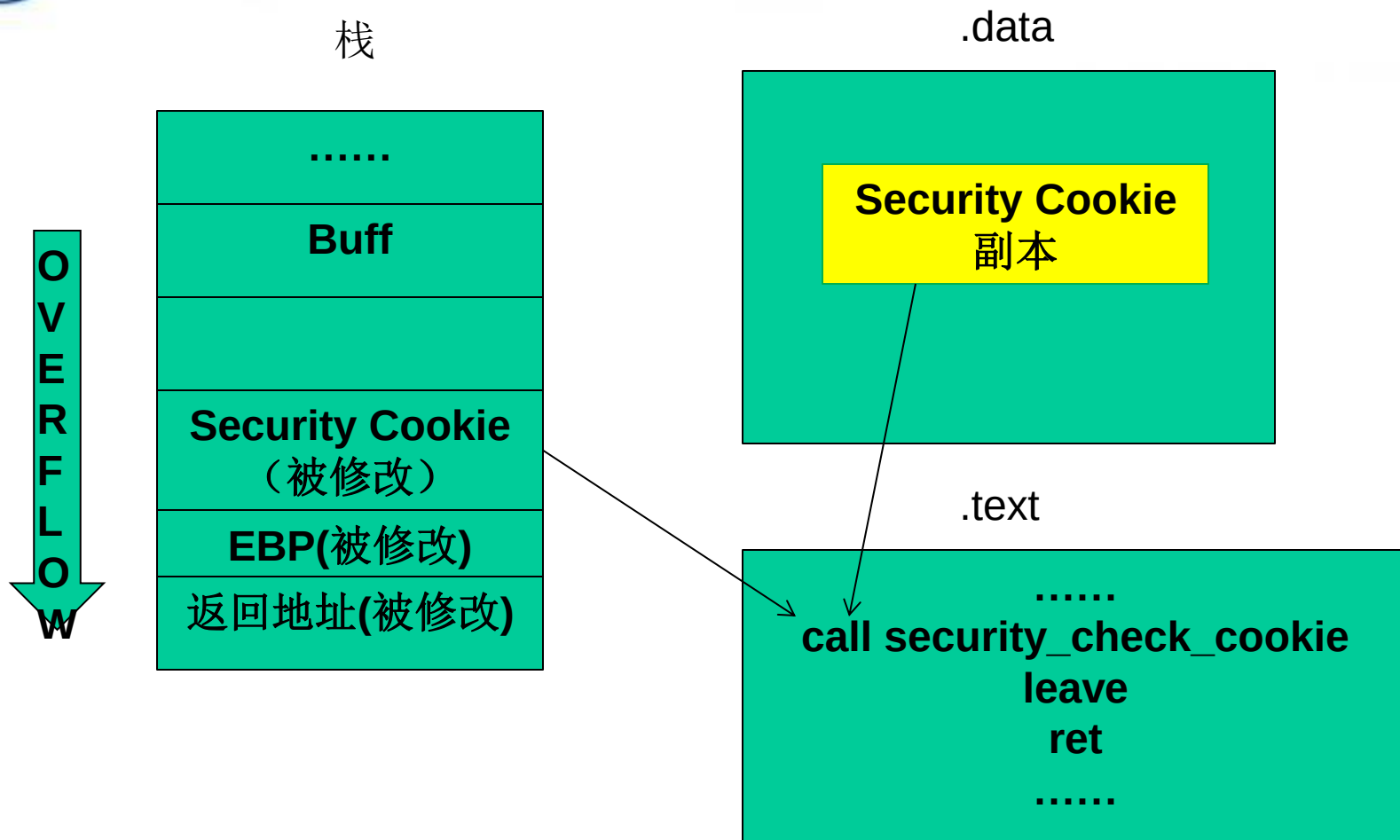
当栈中发生溢出时，首先被淹没的就是**Security Cookie**，之后才是**EBP**和返回地址。在函数返回之前，系统将执行一个额外的安全验证操作，被称做**Security check**。

在**Security Check**的过程中，系统将对两个**Security Cookie**的值，如果不吻合，那么说明栈帧的**Security Cookie**已经被破坏，发生了溢出

当检测到栈中发生溢出时，系统将进入异常处理流程，函数不会正常返回，**ret**指令也不会被执行



Windows安全机制概述



GS保护机制的工作原理



Windows安全机制概述

2) **DEP**(Data Execution Protection,数据执行保护)将数据部分标识为不可执行,阻止了栈、堆和数据节中攻击代码的执行。

溢出攻击的根源在于现代计算机对数据和代码没有明确区分这一先天缺陷,就目前看来重新去设计计算机体系结构基本上是不可能的,我们只能靠向前兼容的修补来减少溢出带来的损害。



Windows安全机制概述

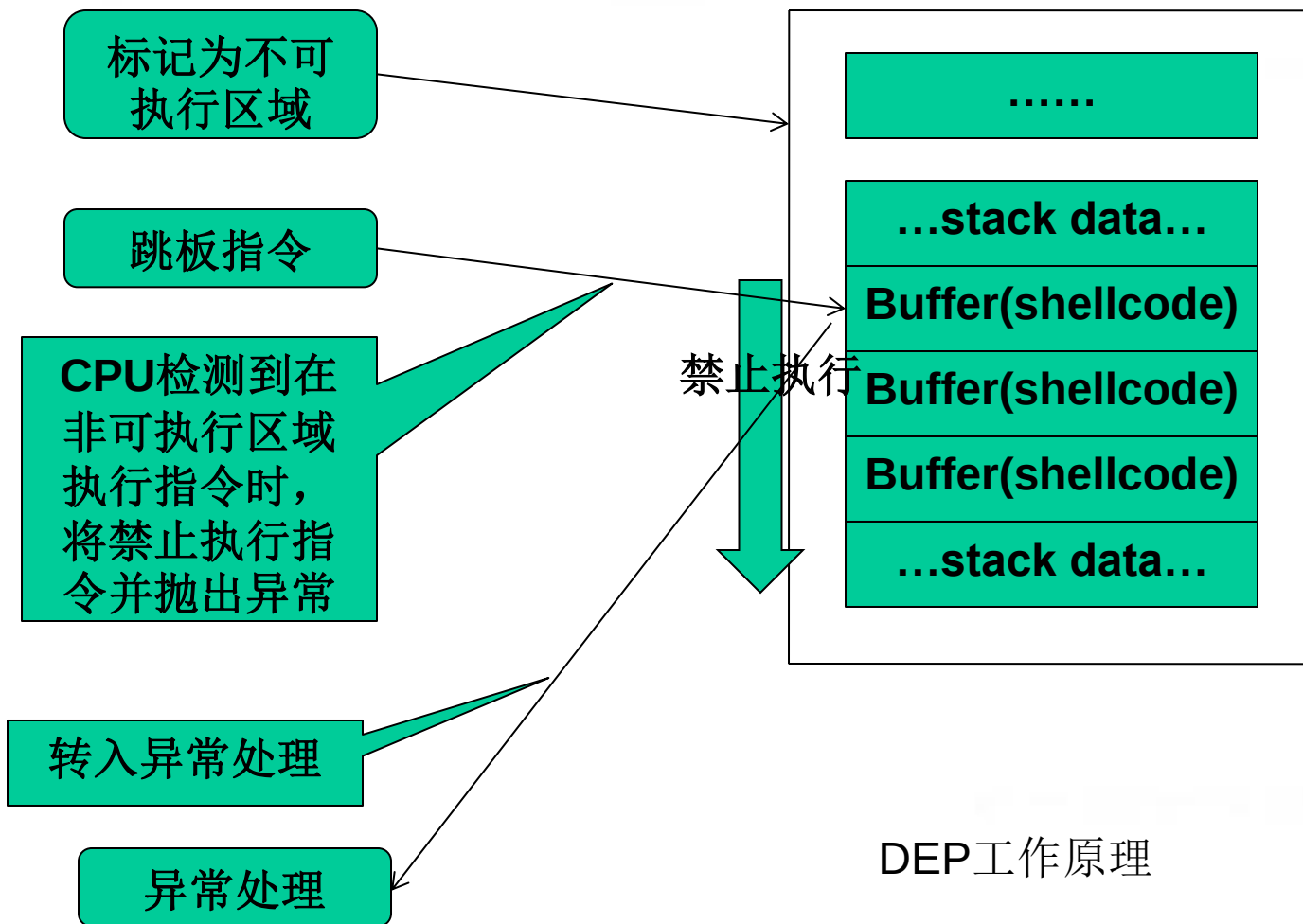
DEP的基本原理是将数据所在内存页标识为不可执行，当程序溢出成功转入shellcode时，程序会尝试在数据页面上执行指令，此时CPU就会抛出异常，而不是去执行恶意指令。

DEP的主要作用就是阻止数据页（如默认的堆页、各种堆栈页及内存池页）执行代码。

微软从Windows XP SP2开始提供这种技术支持。



Windows安全机制概述





Windows安全机制概述

DEP根据实现机制不同可以分为：软件DEP和硬件DEP。

软件DEP就是我们之前介绍的SafeSEH。是Windows利用软件模拟实现DEP，对操作系统提供一定的保护。

而硬件DEP才是真正意义的DEP，它需要CPU的支持。AMD称之为No-Execute Page-Protection(NX),Intel称之为Execute Disable Bit(XD)，两者功能及工作原理在本质上是相同的。



Windows安全机制概述

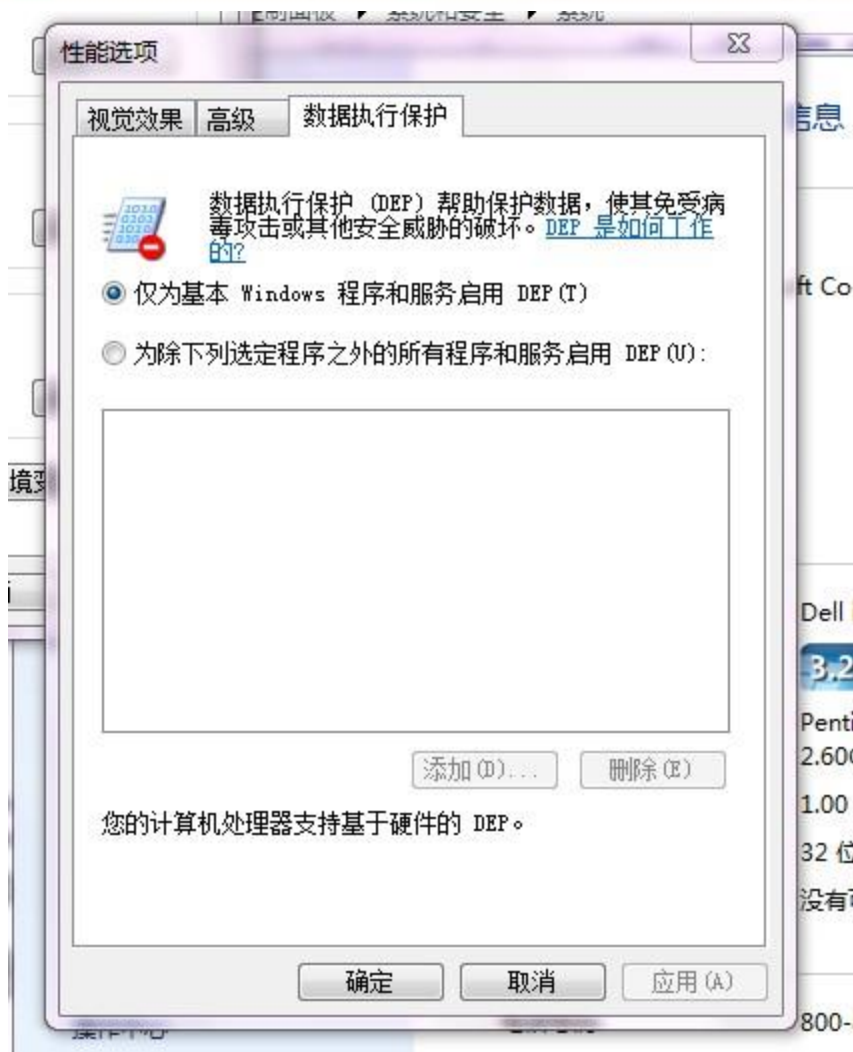
操作系统通过设置内存页的NX/XD属性标记，来指明不能从该内存执行代码。

为了实现这个功能，需要在内存的页面表(Page Table)中假如一个特殊的标识位(NX/XD)来标识是否允许在该页上执行命令，0表示允许，1表示禁止。

大家可以通过下面方法来看CPU是否支持硬件DEP。右键点击我的电脑->属性->高级系统设置->性能->数据保护选项卡。



Windows安全机制概述





Windows安全机制概述

3) 堆中加入了**Heap Cookie**、**Safe Unlinking**等一系列的安全机制，为原本就困难重重的堆溢出增加了更多的限制。

PEB(进程环境块) random: 在堆攻击中，PEB中的函数指针是绝佳的目标。微软在Windows XP SP2之后不再使用固定的PEB基址，而是使用具有一定随机性的PEB基址，PEB随机化之后就影响了对PEB中函数的攻击。



Windows安全机制概述

Safe Unlink:在原来卸载free list的堆块时，代码可能是这样的：

```
int remove(ListNode * node)
{
    node->blink->flink=node->flink;
    node->flink->blink=node->blink;
    return 0;
}
```

blink和**flink**代表前链接和后链接，这样就有一个隐患，当堆溢出成功时，能够伪造出一个节点，节点的前后**link**指向是堆外的地址，那么卸载时就会链接到堆外，产生漏洞。



Windows安全机制概述

改进后会进行验证，以防止发生漏洞:

```
int safe_remove(ListNode * node)
{
    if((node->blink->flink==node)&&
        (node->flink->blink==node))
    {
        node->blink->flink=node->flink;
        node->flink->blink=node->blink;
        return 1;
    }
    else
    {
        return 0;//链表被破坏
    }
}
```



Windows安全机制概述

heap cookie:与栈中的security cookie类似, 微软在堆中也引入了**cookie**, 用于检测堆溢出的发生。**Cookie**被布置在堆首部分原堆块的**segment table** 的位置, 占一个字节。

元数据加密: 微软在Windows Vista及后续版本的操作系统中开始使用该安全措施。块首中的一些重要数据在保存时会与一个4字节的随机数进行异或运算, 在使用这些数据时候需要再进行一次异或运算来还原, 这样我们就不能直接破坏这些数据了, 以达到保护堆的目的。



Windows安全机制概述

4) **ASLR**(Address space layout randomization,加载地址随机)技术通过对系统关键地址的随机化,使得经典堆栈溢出手段失效。

纵观前面介绍的所有漏洞利用方法都有一个共同的特征: 需要确定一个明确的跳转地址

ASLR的概念在xp时就已经提出来了, 只不过功能很有限, 在Vista之后的操作系统才真正的发挥作用, 它包含了以下几个部分。



Windows安全机制概述

映像随机化：映像随机化是PE文件映射到内存时，对其加载的虚拟地址进行随机化处理，这个地址是在系统启动时确定的，系统重启后这个地址会变化。

堆栈随机化：这项措施是在程序运行时随机的选择堆栈的基址，与映像基址随机化不同的是堆栈的基址不是在系统启动的时候确定的，而是在打开程序的时候确定的。

也就是说同一个程序任意两次运行时的堆栈基址都是不同的，进而各个变量在内存中的位置也是不确定的。



Windows安全机制概述

PEB与TEB(线程环境块): PEB与TEB随机化在Windows XP SP2中就已经引入了, 微软在XP SP2之后不再使用固定的PEB基址0x7FFDF000和TEB基址0x7FFDE000, 而是使用具有一定随机性的基址。

但是其随机化的程度不是很好, 而且即使做到了完全随机, 依然还是可以通过其他方法获取到当前进程的PEB和TEB。



北京邮电大学

谢谢观赏！
Thanks!

北京邮电大学

BEIJING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS